

Del III: Skiskyting og datastrukturer (180p)

Del 3 av eksamen er programmeringsoppgaver. Denne delen inneholder **11 oppgaver** som til sammen gir en maksimal poengsum på 180 poeng og teller **ca. 60 %** av eksamen. Hver oppgave gir *maksimal poengsum* på **5 - 25 poeng** avhengig av vanskelighetsgrad og arbeidsmengde.

Den utdelte koden inneholder kompilerbare (og kjørbare) .cpp- og .h-filer med forhåndskodede deler og en full beskrivelse av oppgavene i del 3 som en PDF. Etter å ha lastet ned koden står du fritt til å bruke et utviklingsmiljø (VS Code) for å jobbe med oppgavene.

Du kan laste ned .zip-filen med den utdelte koden fra Inspira. I neste seksjon finner du en beskrivelse av hvordan du gjør dette. Før du leverer eksamen, må du huske å laste opp en .zip-fil med den utdelte koden og endringene du har gjort når du har løst oppgavene.

Nedlasting og oppsett av den utdelte koden

De fem stegene under beskriver hvordan du kan laste ned og sette opp den utdelte koden. En bildebeskrivelse av stegene kan du se i Figur 1g.

1. Last ned .zip-filen fra Inspira.
2. Pakk ut .zip-filen ved å trykke på Extract.
3. I vinduet som kommer opp skriver du inn «C:\temp».
4. Åpne mappen som ligger i C:\temp i VS Code ved å velge:

File → Open Folder ...

5. Kjør følgende kommando i VS Code:

`Ctrl+Shift+P → TDT4102: Create project from TDT4102 template → Configuration only`

Sjekkliste ved tekniske problemer

Hvis prosjektet du oppretter med den utdelte koden ikke kompilerer (før du har lagt til egne endringer) bør du rekke opp hånden og be om hjelp. Mens du venter kan du prøve følgende:

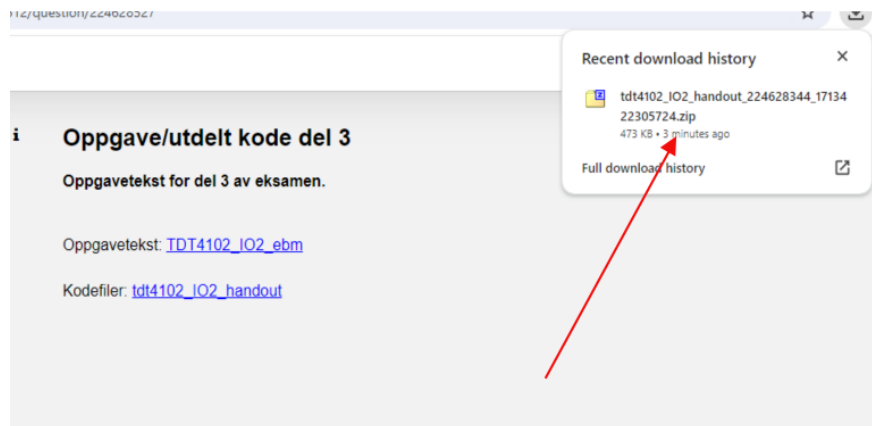
1. Sjekk at prosjektmappen er lagret i mappen C:\temp.
2. Sjekk at navnet på mappen **IKKE** inneholder norske bokstaver (Æ, Ø, Å) eller mellomrom. Dette kan skape problemer når du prøver å kjøre koden i VS Code.
3. Sørg for at du er inne i riktig mappe i VS Code.
4. Kjør følgende TDT4102-kommando:

`Ctrl+Shift+P → TDT4102: Create project from TDT4102 template → Configuration only`

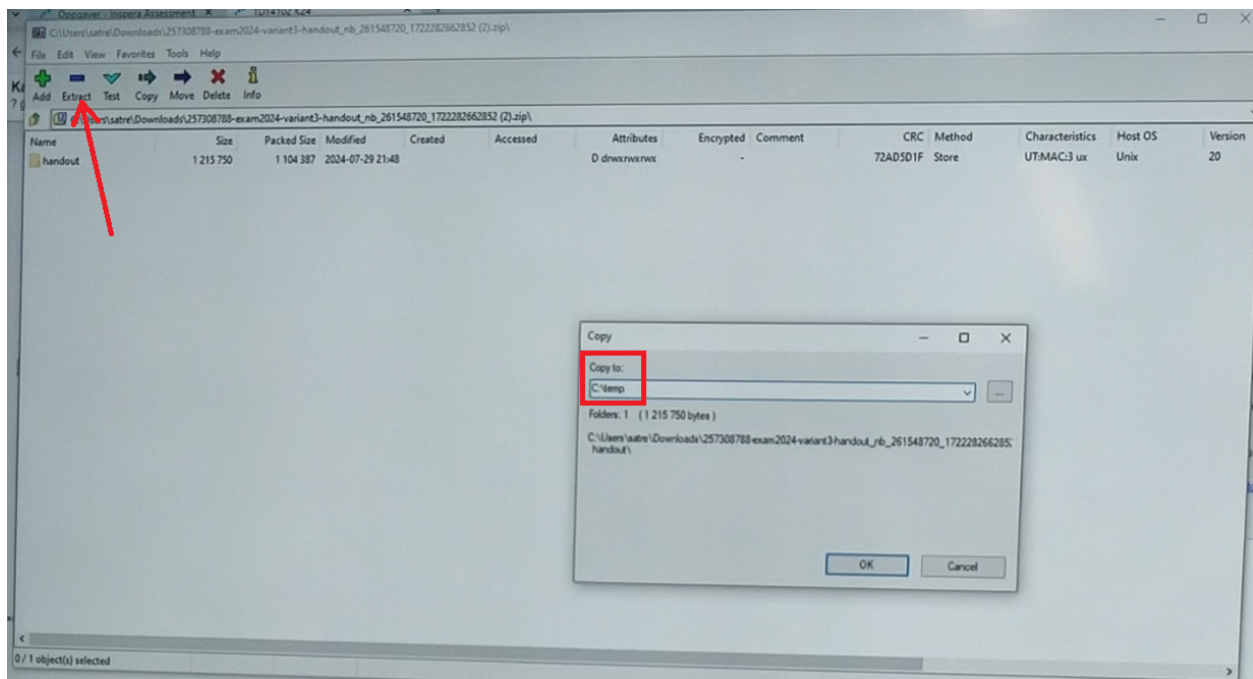
5. Prøv å kjøre koden igjen (F5, Ctrl+F5, eller Fn+F5).
6. Hvis det fortsatt ikke fungerer, lukk VS Code vinduet og åpne det igjen. Gjenta deretter steg 5-6.



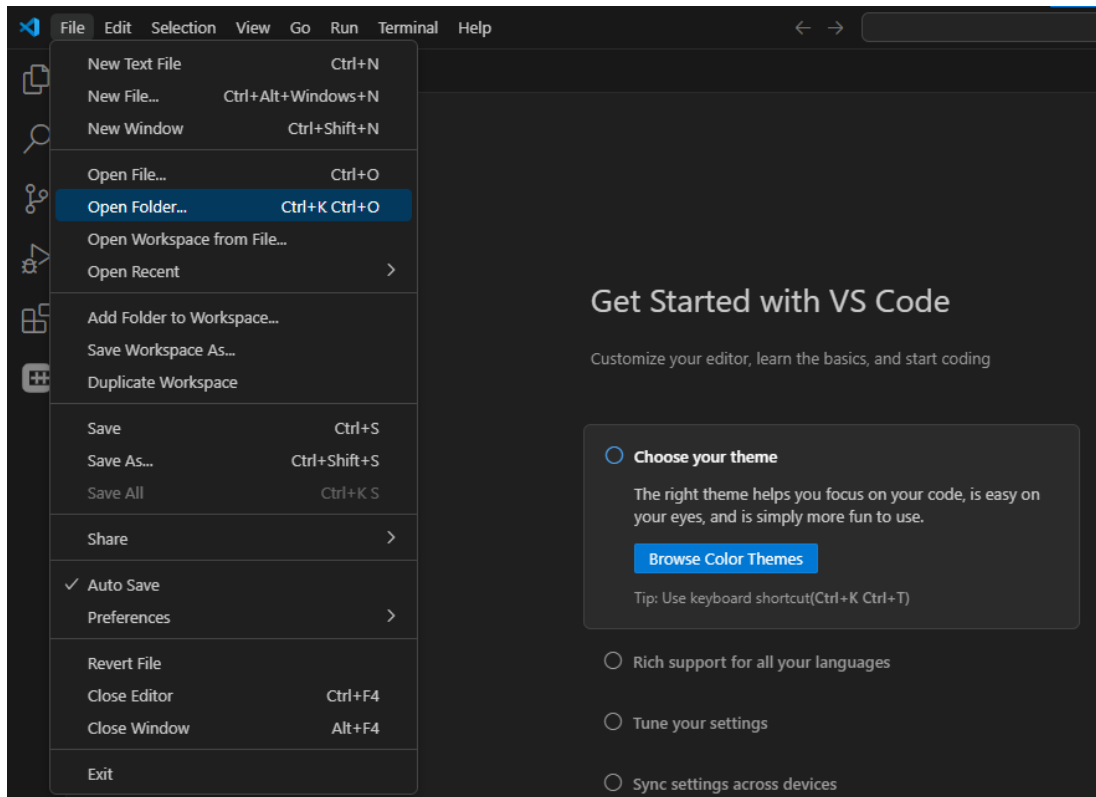
(a) Steg 1



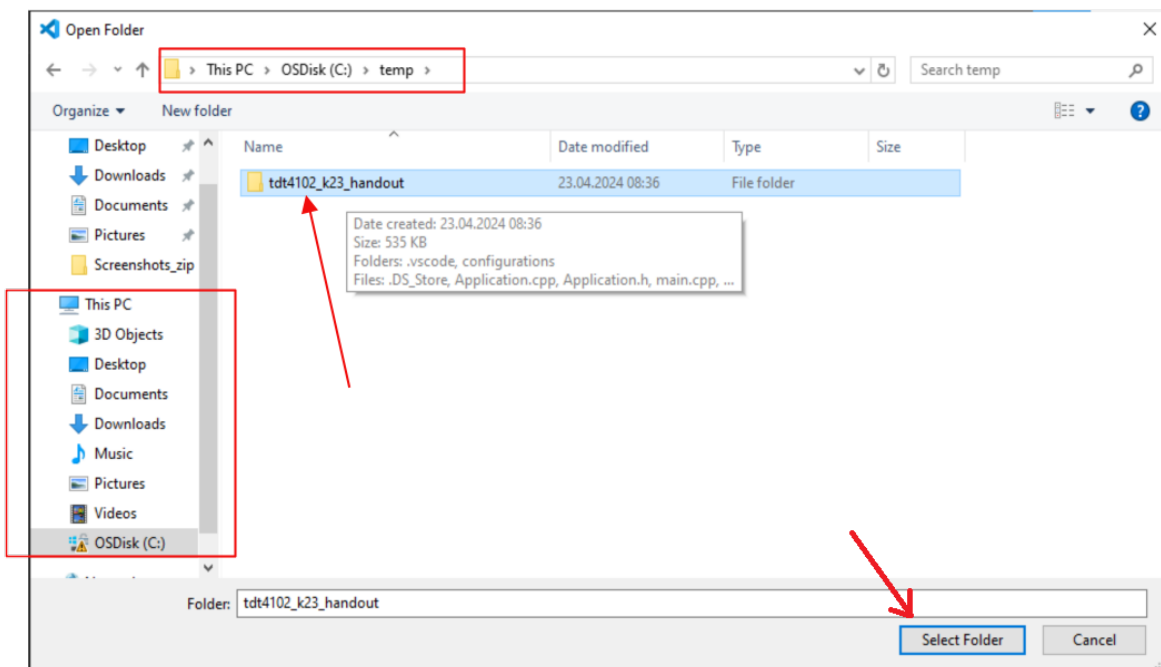
(b) Steg 1



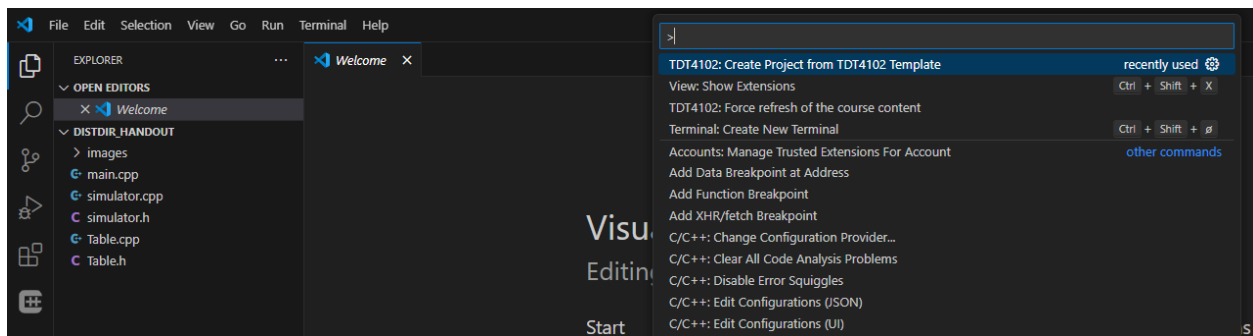
(c) Steg 2 og 3: Skriv inn «C:\temp» i vinduet og klikk OK.



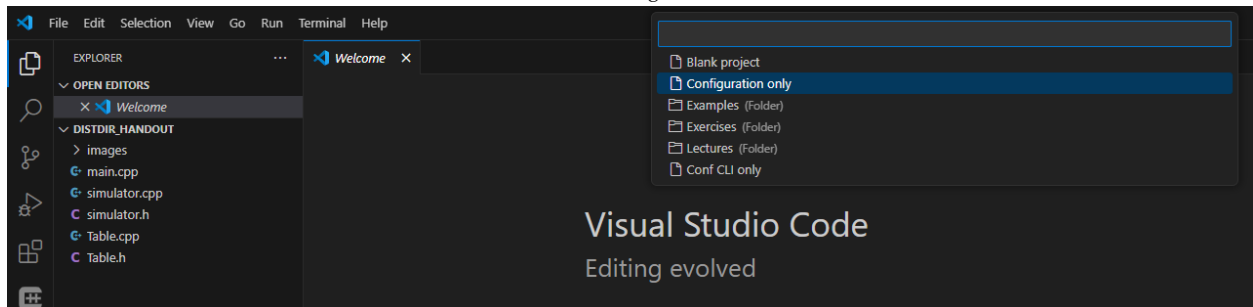
(d) Steg 4



(e) Steg 4



(f) Steg 5



(g) Steg 5

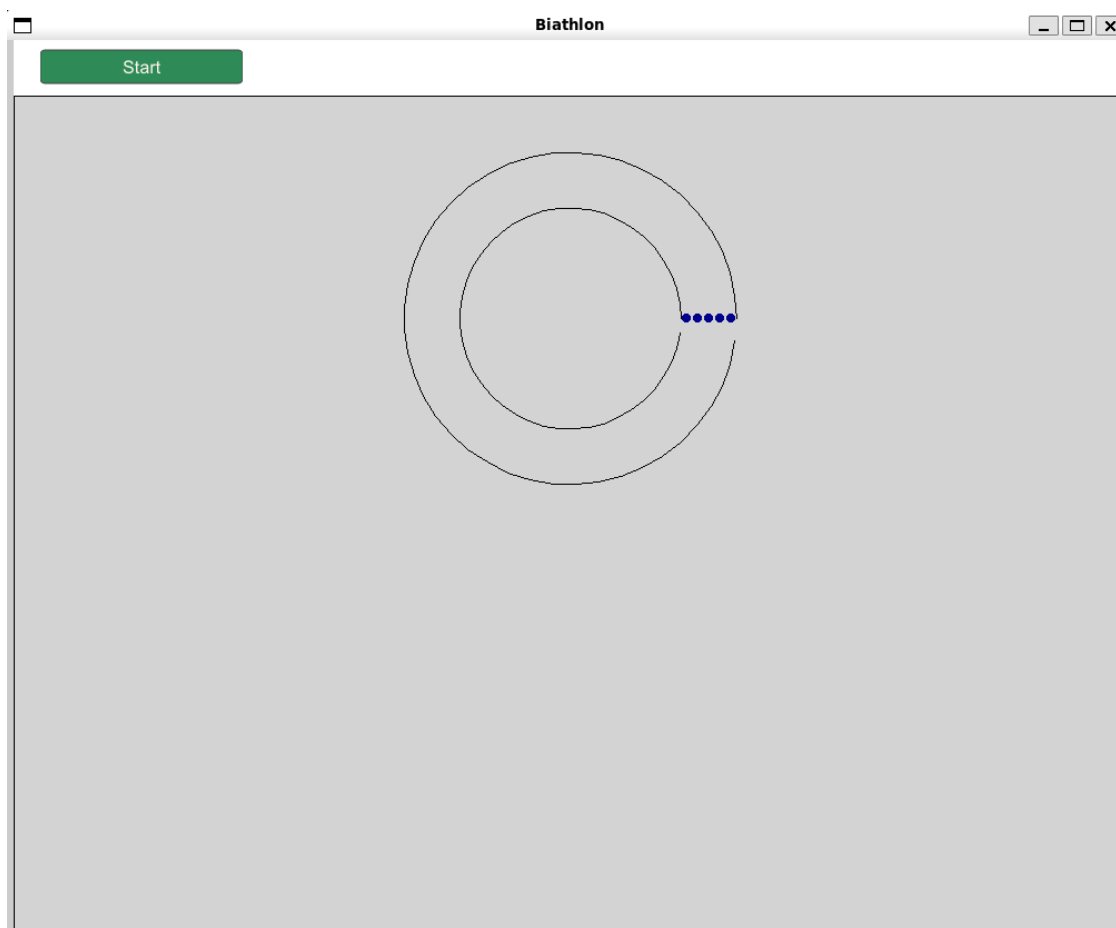
Introduksjon

Del 3 er delt inn i to hovedbolker:

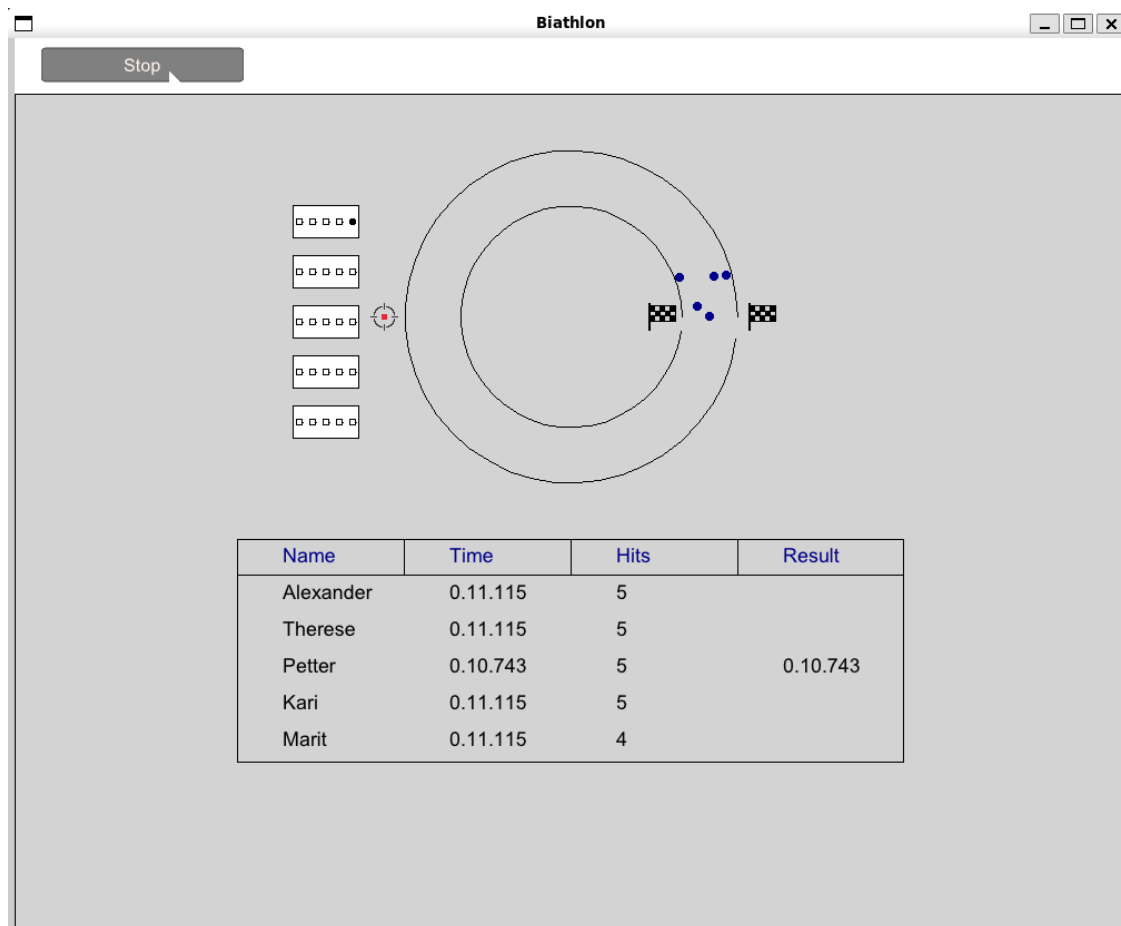
- En skiskytter-simulering som produserer data.
- En tabellstruktur (tenk regneark som i Excel) som viser frem dataen og holder oversikt over resultatene.

Figur 2 viser hvordan kjøring av den utdelte koden ser ut. Figur 3 viser hvordan kjøring av koden skal se ut etter at alle oppgavene er løst.

Simulasjonen kontrolleres av et Simulator-objekt som har en liste med alle Skier-objektene. I tillegg har Simulator en unique_ptr til et Stopwatch-objekt som registrerer tiden, og et Table-objekt for resultatene.



Figur 2: Skjermbilde av kjøring av den utdelte koden. I animasjonsvinduet ser man en funksjonsbar bestående av en Start/Stopp-knapp og to sirkler som skal illustrere en skiløype.



Figur 3: Skjerm bilde av kjøring av koden når alle oppgavene er løst.

Tabellen er generell, slik at hver celle i teorien kan inneholde en vilkårlig datatype, men tabellen er initialisert med `std::string` i simulasjonen. Et `Table`-objekt lagrer verdiene i en to-dimensjonal liste kalt `data`. Hver celle i tabellen består av en `TableData`-struktur. For å holde kontroll på hvilke kolonner som finnes i tabellen, lagres i tillegg kolonnenavnene i listen `columnTitles`. Denne listen inneholder alle de ulike strengene man kan finne i feltene `TableData::columnTitle` i `data`. Under kan du se definisjoner for `TableData`-strukturen og deler av `Table`-klassen.

```
template <typename T>
struct TableData
{
    string columnTitle;
    T value;
};

class Table
{
public:
    void draw(AnimationWindow &window, int height, int width, Point topLeftCorner);
    void storeTable(filesystem::path filepath);
```

```

// (...)
// more functionality below for
// adding and updating values

private:
// (...)

vector<string> columnTitles;
vector<vector<TableData<string>>> data;
};

```

Fordelen med denne måten å lagre cellene på er at radene i data kun inneholder en kolonne dersom den har en verdi. Det betyr at dersom det blir lagt til en ny kolonne, trenger vi ikke gå gjennom hele tabellen og legge til NULL-verdier for radene som allerede eksisterer. Sagt på en annen måte, lagrer vi ikke NULL-verdier hvis de ikke eksplisitt blir lagt inn.

I Figur 4, Figur 5 og Figur 6 kan du se et eksempel på hvordan dette kan henge sammen.

	A	B	C	D
1	Navn	Studie	Retning	Bosted
2	Henrik	Sykepleie		Bergen
3	Ola			Trondheim
4	Kari	Psykologi		
5	Anne	Veterinær	Smådyr	Ås

Figur 4: Eksempel på en tilfeldig tabell. Noter deg at ikke alle feltene er fylt inn.

```

[0] [1] [2] [3]
{"Navn", "Studie", "Retning", "Bosted"}

```

Figur 5: Hvordan listen `columnTitles` kan se ut for tabellen i Figur 4. Tallene over listen skal vise indexen til de ulike verdiene.

```

{
  [0] [1] [2] [3]
  {"Navn": "Henrik", "Studie": "Sykepleie", "Bosted": "Bergen"},
  {"Navn": "Ola", "Bosted": "Trondheim"},
  {"Navn": "Kari", "Studie": "Psykologi"},
  {"Navn": "Kari", "Studie": "Veterinær", "Retning": "Smådyr", "Bosted": "Ås"}
}

```

Figur 6: Hvordan den 2-dimensjonale listen `data` kan se ut for tabellen i Figur 4. Tallene over listen skal vise indeksen til de ulike verdiene. Noter deg at ikke alle vektorene er like lange. Noter deg også at alle radene er sortert i samme rekkefølge som `columnTitles`. Det vil si at 'Navn' alltid kommer først og 'Bosted' alltid kommer sist.

Din oppgave er å utvide og legge til funksjonalitet i den utdelte koden. I .zip-filen finner du et kodeskjelett med tydelig markerte oppgaver.

Hvordan besvare oppgavene

Hvis du synes noen av oppgavene er uklare kan du oppgi hvordan du tolker dem og de antagelsene du gjør for å løse oppgaven som kommentarer i koden du leverer.

Oppgaveteksten

Oppgavetekstene vil ha følgende struktur:

Første del inneholder motivasjon, bakgrunnsinformasjon og hvordan koden er satt sammen for oppgaven.

Neste del er en tekstboks som inneholder oppgaven og spesifikke krav.

Siste del forklarer resultatet du kan forvente når du har fullført oppgaven.

Koden

Hver oppgave i del 3 har en unik kode for å gjøre det lettere for deg å vite hvor du skal fylle inn svarene dine. Koden er på formatet <T><siffer> (TS), der sifrene er mellom 1 og 11 (T1 - T11). For hver oppgave vil man finne to kommentarer som skal definere henholdsvis begynnelsen og slutten av svaret ditt. Kommentarene er på formatet: // BEGIN: TS og // END: TS.

For eksempel kan en oppgave se slik ut:

```
// TASK T1
bool foo(int x, int y) {
// BEGIN: T1
// Write your answer to assignment T1 here, between the // BEGIN: T1
// and // END: T1 comments. You should remove any code that is
// already there and replace it with your own.

return false;

// END: T1
}
```

Etter at du har implementert løsningen din bør du ende opp med følgende:

```
// TASK T1
bool foo(int x, int y) {
// BEGIN: T1
// Write your answer to assignment T1 here, between the // BEGIN: T1
// and // END: T1 comments. You should remove any code that is
// already there and replace it with your own.
    return (x + y) % 2 == 0;
// END: T1
}
```

Det er veldig viktig at alle svarene dine føres mellom slike par av kommentarer. Dette er for å støtte sensurmekanikken vår. Hvis det allerede er skrevet kode mellom BEGIN- og END-kommentarene til en oppgave i det utdelte kodeskjelettet, så kan du, og ofte bør du, erstatte denne koden med din egen implementasjon. All kode som står utenfor BEGIN- og END-kommentarene **SKAL** du la stå som den er.

Merk: Du skal **IKKE** fjerne BEGIN- og END-kommentarene.

Tips: Trykker man CTRL+SHIFT+F og søker på BEGIN: får man snarveier til starten av alle oppgavene listet opp i utforskervinduet slik at man enkelt kan hoppe mellom oppgavene. For å komme tilbake til det vanlige utforskervinduet kan man trykke CTRL+SHIFT+E.

Hvordan levere del 3

Når du er ferdig med oppgavene og er klar til å levere skal du laste opp alle .h- og .cpp-filene i hoved-mappen som en .zip-fil i Inspira. Du står fritt til å bruke den innebygde funksjonen i VS Code til å lage .zip-filen. Disse filene inkluderer:

- Table.h
- Table.cpp
- Simulator.h
- Simulator.cpp
- main.cpp

Oppgavene

1. (5 points) T1: Start simulasjonen

Start-knappen trenger tilgang til variablene `running` og `watch` i `Simulator`-klassen for å kunne kjøre simulasjonen. Vi ønsker derimot ikke å gjøre variablene offentlige, hverken ved å gjøre dem `public` eller gjennom get-funksjoner. Din oppgave er å finne en annen måte å gi `ToggleButton`-klassen tilgang til disse variablene.

I `simulator.h` filen:

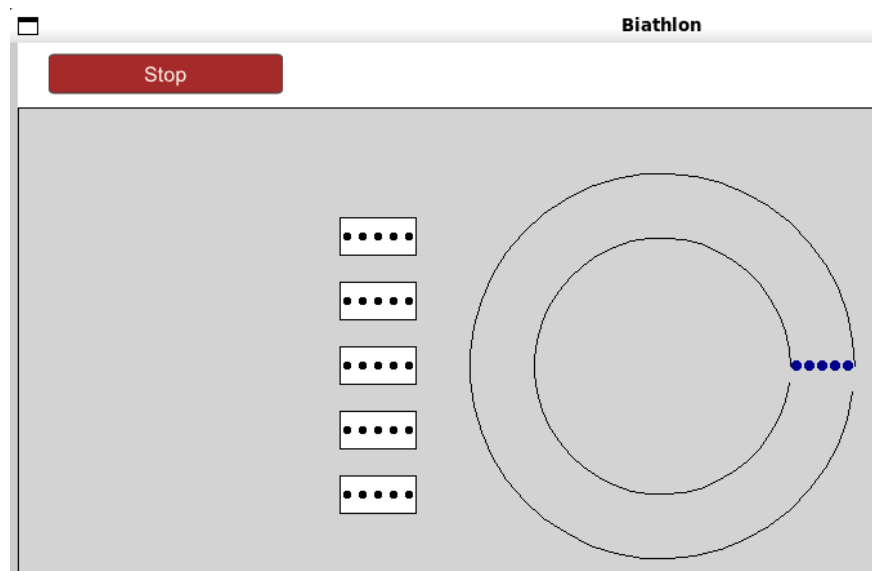
- Fjern eller kommenter ut kodelinjen
`#define START_BUTTON_NO_ACCESS_TO_RUNNING_AND_WATCH`
i `simulator.h`.
- Gi `ToggleButton`-klassen tilgang til variablene `running` og `watch` uten å endre tilgangsnivået fra `private`.

Tips 1: Du kan legge til kodelinjen `#define START_BUTTON_NO_ACCESS_TO_RUNNING_AND_WATCH` igjen dersom du ikke har løst oppgaven enda og ønsker å fjerne feilmeldingene du får fra kompilatoren.

Tips 2: Dersom du ikke får til denne oppgaven kan du kommentere ut eller fjerne linje 155 i `simulator.h`. Dette vil gi alt i `Simulator` tilgangsnivået `public`, og du kan fortsette på de neste oppgavene uten ulemper.

```
// Remove this part to make everything public if you
// did not manage to complete task T1
// REMOVE
private:
// REMOVE
```

Når oppgave T1 er ferdig skal simuleringen starte når man trykker på Start-knappen. Dette ser man ved at knappen bytter mellom Start-knapp og Stop-knapp når den blir trykket på. I tillegg vil man etter ca. 5 sekunder se at målskivene til utøverne dukker opp når de ankommer skytebanen som vist i Figur 7.



Figur 7: Simulasjonen etter at T1 er implementert og man har trykket på Start-knappen og ventet noen sekunder.

2. (5 points) T2: Oppdater antall treff

Når man nå trykker på Start-knappen vil simulasjonen kjøre, men målskivene (se Figur 7) blir ikke oppdatert når en utøver treffer. Det er fordi hit-funksjonen i Target ikke er implementert. Denne funksjonen skal øke hits-variabelen i Target med 1 hver gang den blir kalt. Du finner definisjonen til Target-klassen i `simulator.h`, som ser slik ut:

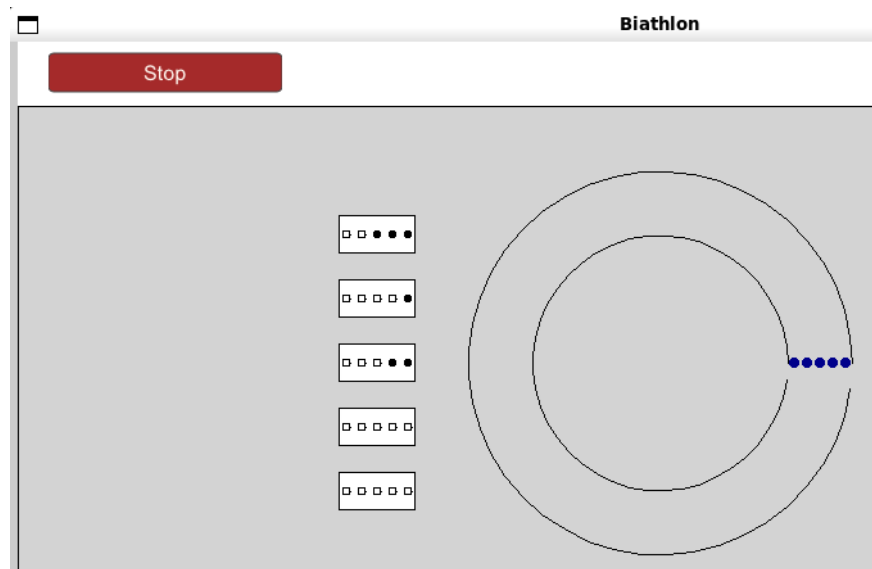
```
class Target
{
public:
    void hit();
    void draw(AnimationWindow *window, Point position, int size);
    int getHits();

private:
    int hits = 0;
};
```

Implementer funksjonen `Target::hit()` i `simulator.cpp`.

- `Target::hit()` skal øke `Target::hits`-variabelen med 1.

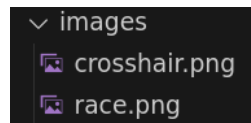
Når oppgave T2 er implementert vil man kunne se at målskivene blir oppdatert etter hvert som utøverne skyter og treffer som vist i Figur 8.



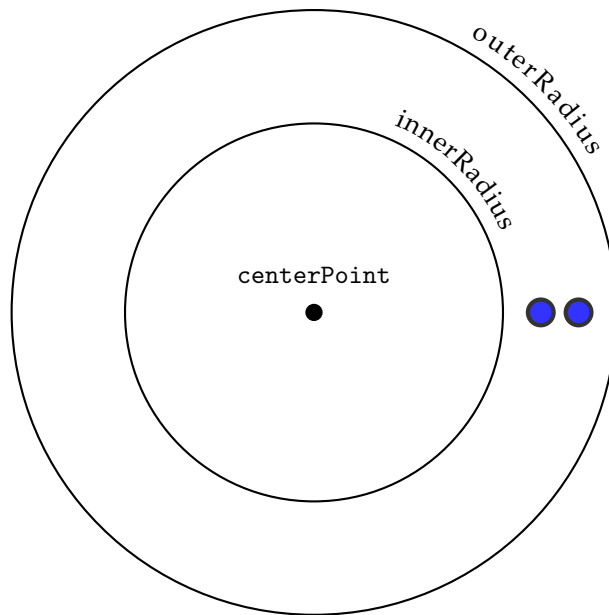
Figur 8: Målskivene vil bli oppdatert når en utøver treffer etter at oppgave T2 er implementert.

3. (10 points) **T3: Marker kartet**

Vi ønsker å markere start, mål og skyteområde på kartet i simulasjonen. I `images`-mappen i den utdelte koden finner du to bilder (.png-filer). Filen `crosshair.png` viser et trådkors og filen `race.png` viser et flagg.



Vi skal nå implementere funksjonen `Simulator::placeImages` som skal bruke bildene til å markere kartet. Denne funksjonen tar 4 parametre: `centerPoint`, `innerRadius`, `outerRadius` og `size`. `size` beskriver størrelsen som skal brukes på bildene. De andre variablene er illustrert i Figur 9, og skal brukes til å plassere bildene på riktig sted.



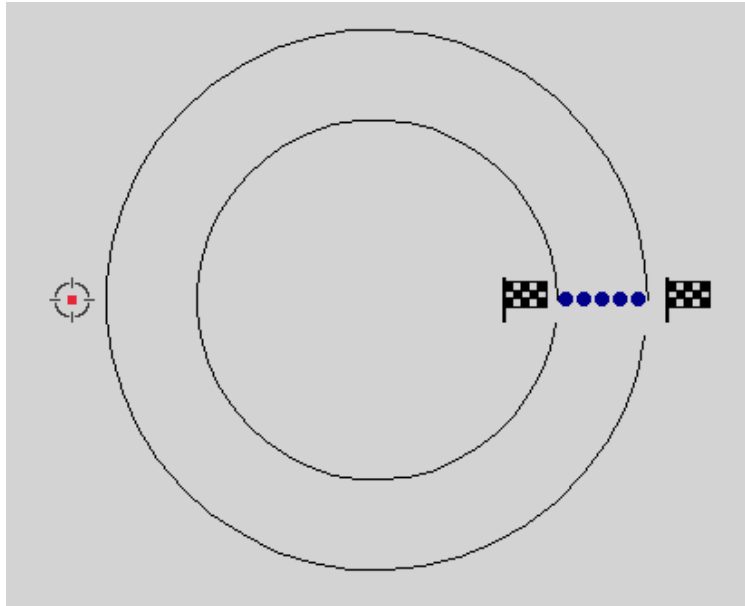
Figur 9: `centerPoint` er midtpunktet vi orienterer fra. `innerRadius` er avstanden fra `centerPoint` til den innerste sirkelen. `outerRadius` er avstanden fra `centerPoint` til den ytterste sirkelen.

Implementer funksjonen `Simulator::placeImages` i `simulator.cpp`.

- Bruk parametrene `centerPoint`, `innerRadius` og `outerRadius` til å tegne flagg ved start og mål, og et trådkors ved skytområdet.
- Bildefilene (.png-filene) som skal brukes ligger i `images`-mappen. Filen `crosshair.png` viser et trådkors og filen `race.png` viser et flagg.
- `size`-parameteren skal brukes til å sette riktig størrelse på bildene.

Hint: Man kan finne informasjon om hvordan man bruker bilder i dokumentasjonen til `AnimationWindow`.

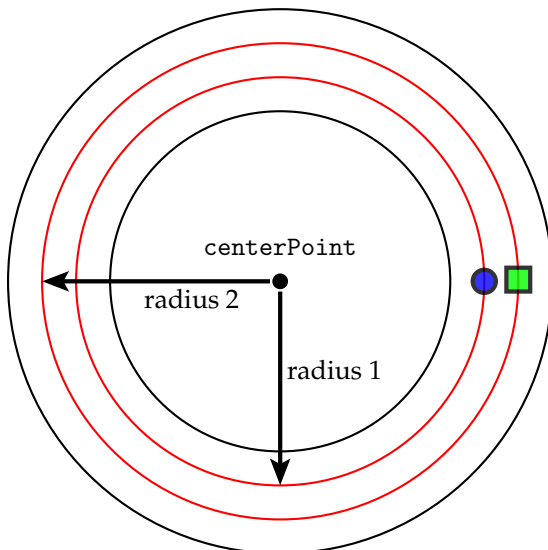
Når oppgave T3 er gjort skal kartet se ut omtrent som kartet i Figur 10. Det tar litt lenger tid før målskivene vises etter denne oppgaven er implementert.



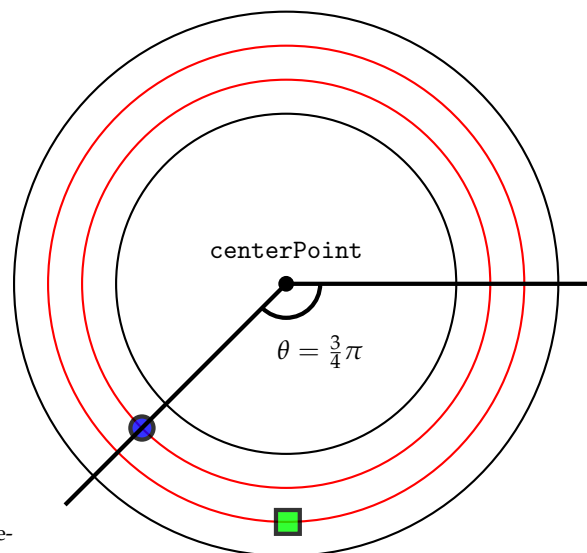
Figur 10: Skjerm bilde av simulasjonen etter at oppgave T3 er implementert, med flagg ved start og mål, og trådkors ved skyteområdet.

4. (20 points) **T4: Få utøverne ut i løypen**

Vi ønsker å kunne visualisere progresjonen til hver enkelt utøver. Siden kartet er gitt som en sirkel, vil posisjonen til en utøver være gitt av en *radius* som forteller hvilken fil de ligger i og en *vinkel* som forteller hvor langt de har kommet. Dette er vist i henholdsvis Figur 11a og Figur 11b.



(a) Radius til to utøvere, der *radius 1* er radiusen til utøveren representert med en sirkel, og *radius 2* er radiusen til utøveren representert med en firkant.



(b) Vinkelen til en utøver på en bane av lengde 1.

Figur 11: Oversikt over løypevariabler.

x -posisjonen til en utøver er dermed gitt av

$$x = x_c + r \cdot \cos \theta,$$

mens y -posisjonen til en utøver er gitt av

$$y = y_c + r \cdot \sin \theta,$$

hvor x_c og y_c representerer `centerPoint` sin x - og y -koordinat, og r er radiusen. θ er progresjonen til utøveren og er gitt av

$$\theta = 2\pi \cdot \frac{\text{position}}{\text{distance}},$$

hvor *distance* er lengden på banen og *position* er posisjonen til utøveren. Disse verdiene finner man henholdsvis i `Map::distance` og ved å bruke `Skier::getPosition()`. `Map`-strukturen er definert i `simulator.h` og er lagret som en variabel i `Simulator`-klassen. `Skier::getPosition` er implementert i `simulator.cpp`.

```
struct Map
{
    double distance;
    double shootingPosition;
};

double Skier::getPosition() { return position; }
```

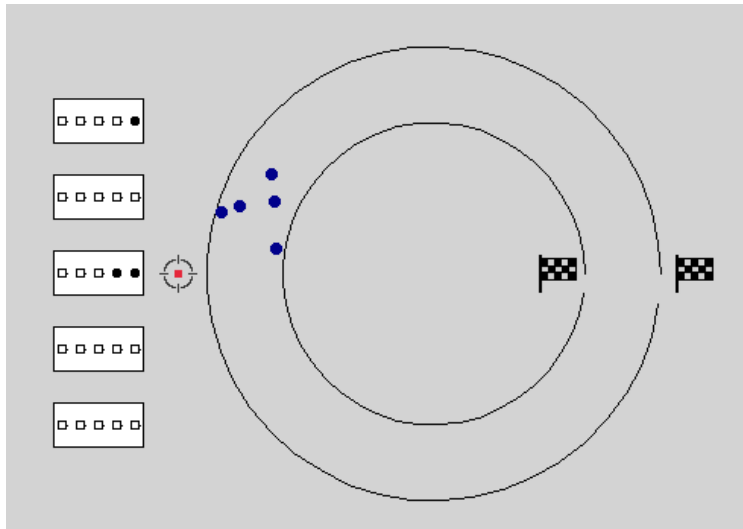
Implementer funksjonen `Simulator::calculateSkierPosition` i `simulator.cpp`.

- Funksjonen skal returnere et `TDT4102::Point` som illustrerer hvor i løypen utøveren befinner seg.
- x -posisjonen til en utøver er gitt av $x = x_c + r \cdot \cos \theta$.
- y -posisjonen til en utøver er gitt av $y = y_c + r \cdot \sin \theta$.

x_c og y_c representerer `centerPoint` sin x - og y -koordinat, og r er radiusen.

θ er progresjonen til utøveren og er gitt av $\theta = 2\pi \cdot \frac{\text{position}}{\text{distance}}$.

Når oppgave T4 er implementert skal utøverne bevege seg rundt på kartet under simuleringen som i Figur 12.



Figur 12: Skjerm bilde av simuleringen etter at oppgave T4 er implementert.

5. (20 points) T5: Fyll tabellen

Vi ønsker en tabell som viser hvordan utøverne ligger an i rennet. Denne tabellen er delvis implementert, men mangler mulighet for å legge til rader. Implementer template-funksjonen `Table::addRow` i `table.h`. Denne funksjonen skal ta inn en rad i form av en vector av `TableData` som parameter. `TableData`-strukturen er deklarert i `table.h` og ser slik ut:

```

1  template <typename T>
2  struct TableData
3  {
4      string columnName;
5      T value;
6  };

```

`Table`-klassen er definert i `table.h` og har medlemsvektorer `columnTitles` og `data`, hvor henholdsvis kolonnenavn og data er lagret:

```

1  vector<string> columnTitles;
2  vector<vector<TableData<string>>> data;

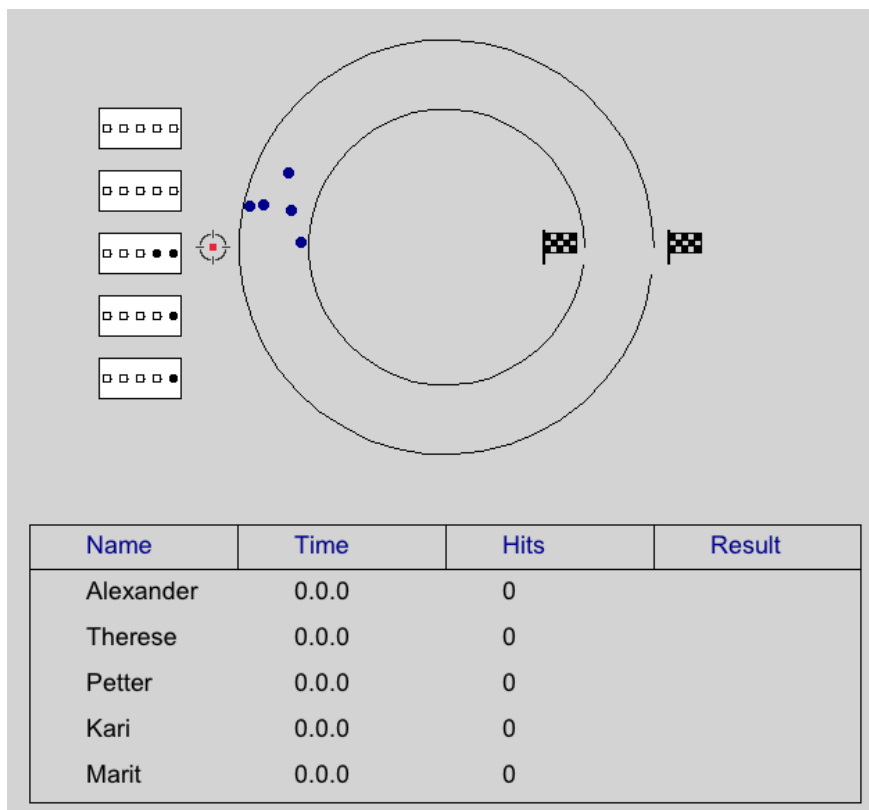
```

Definer og implementer template-funksjonen `Table::addRow` i `table.h`. Funksjonen tar inn en vector av `TableData` som parameter. Funksjonen har ingen returverdi. I tillegg:

- Fjern eller kommenter ut kodelinjen `#define ADD_ROW_NOT_IMPLEMENTED` i `table.h`.
- Legg til kolonnenavnet i `Table::columnTitles`-vektoren hvis det ikke allerede eksisterer.
- Hele raden skal legges til i `Table::data`-vektoren.

Tips: Du kan legge til kodelinjen `#define ADD_ROW_NOT_IMPLEMENTED` igjen dersom du ikke har løst oppgaven enda og ønsker å fjerne feilmeldingene du får fra kompilatoren.

Når oppgave T5 er gjort skal tabellen være synlig i viduet under simuleringen slik som i Figur 13.



Figur 13: Når oppgave T5 er gjort skal tabellen være synlig i viduet under simuleringen.

6. (20 points) T6: Indeksering

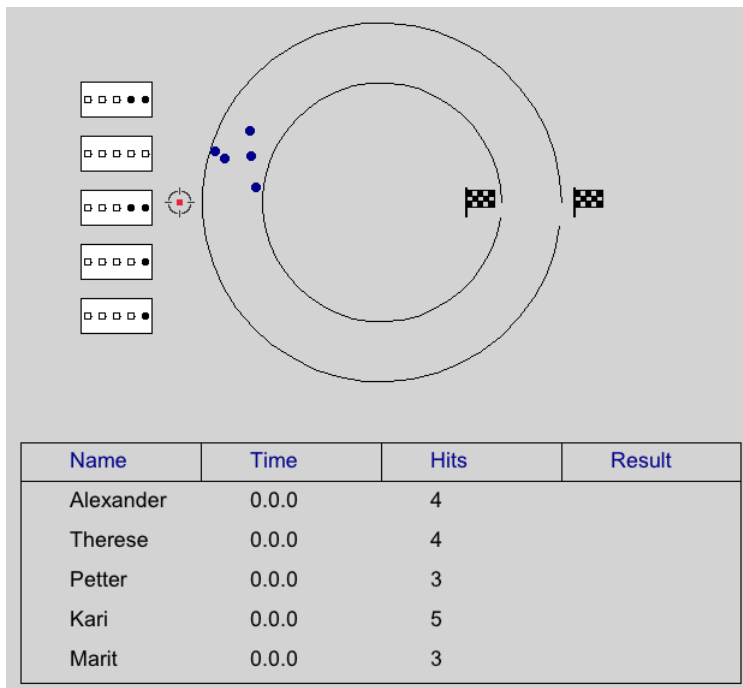
Indeksen til en kolonne i tabellen samsvarer ikke nødvendigvis med indeksen til kolonnen i radene i data. For å finne ut hvor dataen skal plasseres, må vi derfor kunne sammenligne `TableData::columnTitle` med kolonnenavnene i `Table::columnTitles`.

Definer og implementer template-funksjonen `Table::getColumnIndexInRow` i `table.h`. Funksjonen skal ta inn en `std::string` som beskriver kolonnetittel og en vector av `TableData` som tilsvarende en rad i data som parametre. Funksjonen skal returnere en `int` som tilsvarende indeksen til kolonnen i raden. I tillegg

- Fjern eller kommenter ut kodelinjen `#define GET_INDEX_NOT_IMPLEMENTED` i `table.h`.
- Returner indeksen til stedet man kan finne verdien til den gitte kolonnetittelen for denne spesifikke raden.
- Dersom kolonnetittelen vi ser etter ikke finnes i data skal funksjonen returnere -1.

Tips: Du kan legge til kodelinjen `#define GET_INDEX_NOT_IMPLEMENTED` igjen dersom du ikke har løst oppgaven enda og ønsker å fjerne feilmeldingene du får fra kompilatoren.

Etter at oppgave T6 er implementert vil du kunne se at tabellen oppdaterer verdiene i *Hits*-kolonnen etter hvert som utøverne treffer blinken under skyting, som vist i Figur 14.



Figur 14: Skjerm bilde av simulasjonen etter at oppgave T6 er implementert.

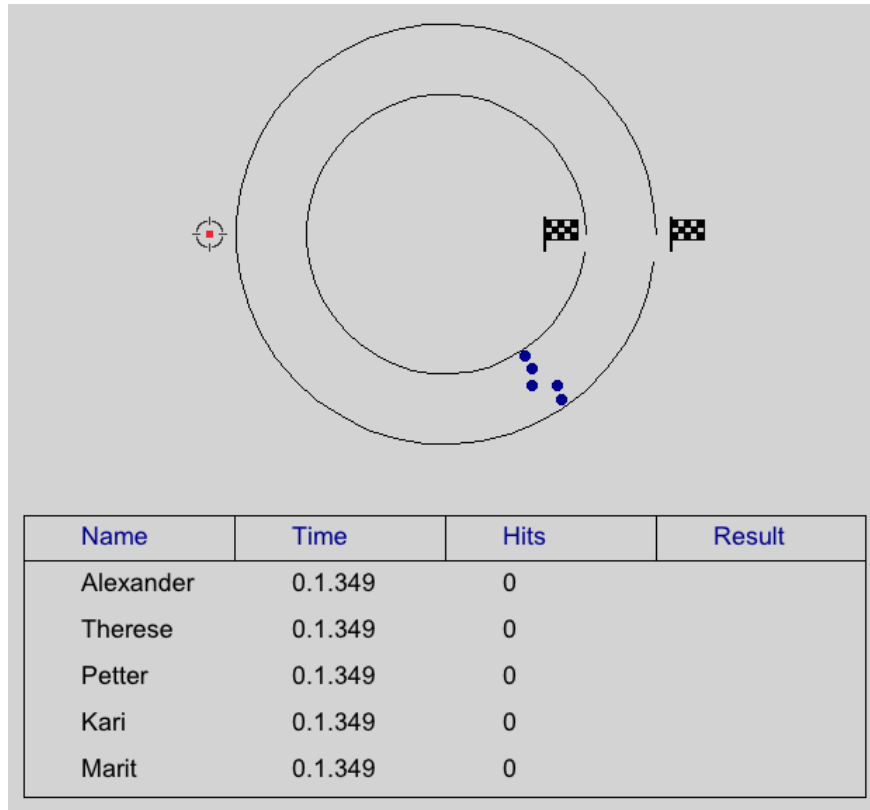
7. (15 points) **T7: Hvert sekund teller!**

Vi ønsker å registrere tidene utøverne bruker på løypa og vise det frem i tabellen i simulasjonen. Funksjonen `Stopwatch::millisecondsToString` tar inn en parameter, `timeInMilliseconds`, som er en `int` som angir hvor mange millisekunder som har gått siden Start-knappen ble trykket på. Vi ønsker å omgjøre denne tiden til et mer leselig format, litt som på en stoppeklokke.

Implementer funksjonen `Stopwatch::millisecondsToString` i `simulator.cpp`.

- Konverter verdien til `timeInMilliseconds` til antall minutter, sekunder, og millisekunder.
- Funksjonen skal returnere en `std::string` på formatet `'minutes.seconds.milliseconds'`.

Når oppgave T7 er gjort vil du kunne se at rundetidene i kolonnen *Time* oppdaterer seg under simuleringen, som vist i Figur 15.



Figur 15: Når oppgave T7 er gjort vil du kunne se at rundetidene i kolonnen *Time* oppdaterer seg under simuleringen.

8. (25 points) **T8: Straffetid**

For å finne det endelige resultatet ønsker vi å kombinere tiden til en utøver med antall treff utøveren hadde ved å legge til straffetid. Slik tabellen er satt opp nå er tidene lagret som en `std::string`. Vi må derfor ha mulighet til å omgjøre strengen til millisekunder for å kunne legge til straffetid, og må gjøre det motsatte av det vi gjorde i forrige oppgave (T7).

Implementer funksjonen `Stopwatch::stringToMilliseconds` i `simulator.cpp`. Funksjonen tar inn en `std::string` på formatet 'minutes.seconds.milliseconds' og skal returnere en `int` som er tiden i millisekunder.

Oppgave T8 kan ikke verifiseres visuelt.

9. (25 points) **T9: Hvem havner på pallen?**

Som nevnt i forrige oppgave (T8) bestemmes det endelige resultatet ved å kombinere tiden til en utøver med antall treff utøveren hadde ved å legge til straffetid. For å finne den endelige tiden til en utøver må vi dermed lese verdien som er lagret i kolonnene *Time* og *Hits*, konvertere begge verdiene fra strenger til tall, regne ut og legge til straffetid, og til slutt konvertere tiden tilbake til en streng som viser den endelige tiden på et leselig format.

Straffetiden per bom er oppgitt i den utdelte koden som konstanten `PENALTY_FACTOR`:

$$\text{PENALTY_FACTOR} = \frac{\text{PENALTY}}{\text{SPEED_SCALE}}. \quad (1)$$

Klassen `Stopwatch` inneholder funksjonene `millisecondsToString` og `stringToMilliseconds` som du implementerte i oppgave T7 og T8. Disse kan brukes til å konvertere tid frem og tilbake mellom strenger og tall. `Simulator`-klassen har en instans av `Stopwatch` kalt `watch` lagret som en `unique_ptr`. `Stopwatch`-klassen er definert i `simulator.h` og ser slik ut:

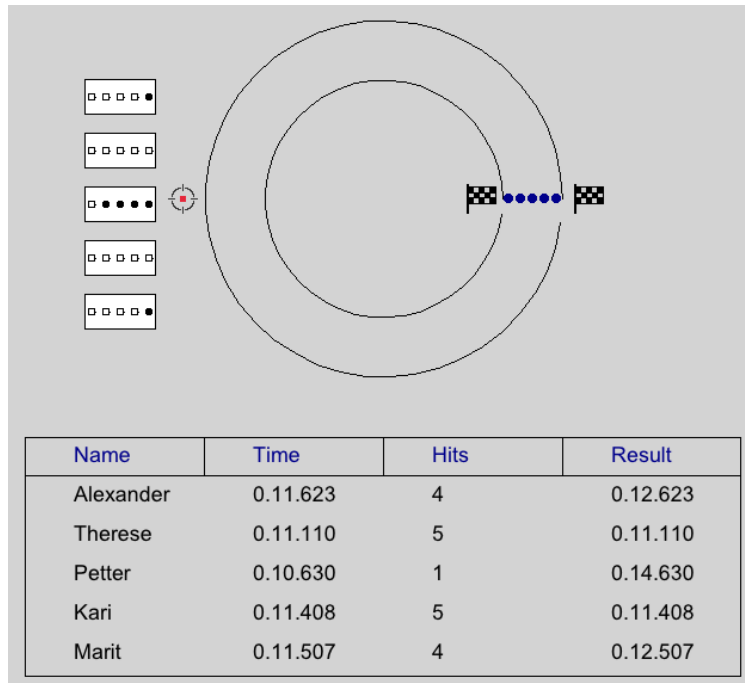
```
class Stopwatch
{
public:
    void toggleWatch();
    std::chrono::duration<int64_t, std::nano> getTime();
    string timeToString(std::chrono::duration<int64_t, std::nano> clock);
    string millisecondsToString(int time);
    int stringToMilliseconds(string time);

private:
    vector<std::chrono::time_point<std::chrono::steady_clock>> timeEvents;
};
```

Implementer funksjonen `Simulator::finaliseResult` i `simulator.cpp`. Funksjonen tar inn en rad fra `Table::data` og skal returnere summen av rundetiden og straffetiden som en `std::string` på formatet 'minutes:seconds:milliseconds'.

- Hent ut indeksen til kolonnen `Time` i raden. Dersom kolonnenavnet ikke finnes skal du returnere strengen 'Time missing'.
- Bruk `Stopwatch`-instansen `watch` i `Simulator` til å konvertere rundetiden til millisekunder.
- Hent ut indeksen til kolonnen `Hits`. Dersom kolonnenavnet ikke finnes skal du returnere strengen 'Hits missing'.
- Legg til straffetiden til rundetiden. Straffetiden per bom finnes i konstanten `PENALTY_FACTOR` (Likning (1)).
- Konverter den endelige tiden (rundetiden + straffetiden) til en `std::string` på formatet 'minutes:seconds:milliseconds'.
- Returner strengen du konstruerte i punktet over.

Når oppgave T9 er gjort vil du kunne se at tabellen oppdaterer *Result*-kolonnen etter hvert som utøverne kommer i mål, som vist i Figur 16.



Figur 16: Skjerm bilde av simulasjonen etter at oppgave T9 er gjort.

10. (15 points) **T10: Operatoroverlasting**

Vi ønsker å forenkle prosessen med å skrive ut `TableData`-strukturer. Vi vil derfor overlaste `<<`-operatoren.

Overlast `<<`-operatoren for `TableData` i `table.h`.

- Funksjonen skal ta inn 2 parametre, en `std::ostream` og en `TableData`-struktur med generell type.
- Vi ønsker å skrive data til strømmen på formatet `'TableData::columnTitle:TableData::value'`.

Oppgave T10 kan ikke verifiseres visuelt.

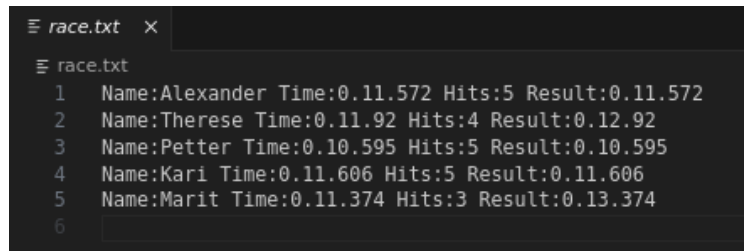
11. (20 points) **T11: Lagre resultatene**

Vi ønsker å lagre resultatene fra simuleringen i en fil for senere bruk.

Implementer funksjonen `Table::storeTable` i `table.cpp`.

- Dersom `filepath` ikke finnes skal funksjonen utløse et passende unntak.
- Data for en utøver skal lagres som en linje i filen.
- Kolonnenavnet og kolonneverdien skal lagres som `columnTitle:value` for hver kolonne, med et mellomrom mellom kolonnene. Du kan se filformatet i Figur 17.

Når oppgave T11 er implementert blir det lagret en fil, `race.txt`, i hovedmappen på formatet vist i Figur 17 når simuleringen er ferdig.

A screenshot of a text editor window titled 'race.txt'. The editor shows a list of five race results, each on a new line. The results are numbered 1 through 5. Each line contains the name of the participant, their time, the number of hits, and the result. The text is as follows:
1 Name:Alexander Time:0.11.572 Hits:5 Result:0.11.572
2 Name:Therese Time:0.11.92 Hits:4 Result:0.12.92
3 Name:Petter Time:0.10.595 Hits:5 Result:0.10.595
4 Name:Kari Time:0.11.606 Hits:5 Result:0.11.606
5 Name:Marit Time:0.11.374 Hits:3 Result:0.13.374
Line 6 is empty.

```
≡ race.txt x
≡ race.txt
1 Name:Alexander Time:0.11.572 Hits:5 Result:0.11.572
2 Name:Therese Time:0.11.92 Hits:4 Result:0.12.92
3 Name:Petter Time:0.10.595 Hits:5 Result:0.10.595
4 Name:Kari Time:0.11.606 Hits:5 Result:0.11.606
5 Name:Marit Time:0.11.374 Hits:3 Result:0.13.374
6
```

Figur 17: Formatet til filen som tabellen blir lagret i.